

Kotlin では、`fun` キーワードを使って独自の関数を宣言することができます。

```
fun hello() {  
    return println("Hello, world!")  
}  
  
fun main() {  
    hello()  
    // Hello, world!  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-function-demo"}

Kotlinでは：

- 関数の引数は、括弧 `()` の中に記述します。
- 各パラメータには型を指定する必要があります。また、複数のパラメータを指定する場合は、カンマ `,` で区切る必要があります。
- 戻り値の型は、関数の括弧 `()` の後に、コロン `:` を挟んで記述します。
- 関数の本体は、中括弧 `{}` で囲んで記述します。
- `return` キーワードは、関数を終了したり、関数から何かを返したりするために使用されます。

関数が有用な値を返さない場合は、戻り値の型と `return` キーワードを省略できます。詳細については、「[戻り値のない関数](#)」をご覧ください。

{style="note"}

c次の例では：

- `x` と `y` は関数の引数です。
- `x` と `y` の型は `Int` です。
- この関数の戻り値の型は `Int` です。
- この関数は、呼び出されると `x` と `y` の和を返します。

```
fun sum(x: Int, y: Int): Int {  
    return x + y  
}  
  
fun main() {  
    println(sum(1, 2))  
    // 3  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-simple-function"}

Kotlinの[コーディング規約](#)では、関数の名前は小文字で始め、アンダースコアを含まないキャメルケースを使用することを推奨しています。

{style="note"}

Named arguments

名前付き引数

コードを簡潔にするため、関数を呼び出す際にパラメータ名を指定する必要はありません。ただし、パラメータ名を指定した方がコードの可読性は高まります。これを**名前付き引数**の使用といいます。パラメータ名を指定する場合、パラメータの順序は自由に決めることができます。

以下の例では、**文字列テンプレート** (\$) を使用してパラメータの値にアクセスし、それらを **String** 型に変換した後、文字列として連結して出力しています。

{style="tip"}

```
fun printMessageWithPrefix(message: String, prefix: String) {
    println("[$prefix] $message")
}

fun main() {
    // パラメータの順序を入れ替えた名前付き引数を使用します
    printMessageWithPrefix(prefix = "Log", message = "Hello")
    // [Log] Hello
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-named-arguments-function"}

Default parameter values

パラメータのデフォルト値

関数のパラメータには、デフォルト値を定義することができます。デフォルト値が設定されているパラメータは、関数を呼び出す際に省略可能です。デフォルト値を宣言するには、型の後に代入演算子 **=** を使用します。

```
fun printMessageWithPrefix(message: String, prefix: String = "Info") {
    println("[$prefix] $message")
}

fun main() {
    // 両方の引数を指定して関数が呼び出される
    printMessageWithPrefix("Hello", "Log")
    // [Log] Hello

    // メッセージパラメータのみを引数として呼び出される関数
    printMessageWithPrefix("Hello")
    // [Info] Hello

    // 名前付き引数を使用し、パラメータの順序を反対にする
}
```

```
    printMessageWithPrefix(prefix = "Log", message = "Hello")
    // [Log] Hello
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-default-param-function"}

デフォルト値を持つ特定のパラメータは、すべて省略するのではなく、スキップすることができます。ただし、最初のパラメータをスキップした後は、それ以降のパラメータはすべて名前を指定する必要があります。

{style="note"}

Functions without return

戻り値を持たない関数

関数が有用な値を返さない場合、その戻り値の型は `Unit` となります。`Unit` は、`Unit` という値のみを持つ型です。関数本体内で、`Unit` が返されることを明示的に宣言する必要はありません。つまり、`return` キーワードを使用したり、戻り値の型を宣言したりする必要はありません：

```
fun printMessage(message: String) {
    println(message)
    // `return Unit` または `return` は省略可能です
}

fun main() {
    printMessage("Hello")
    // Hello
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-unit-function"}

Single-expression functions

単一式関数

コードをより簡潔にするには、単一式関数を使用することができます。たとえば、次の `sum()` 関数は：

```
fun sum(x: Int, y: Int): Int {
    return x + y
}

fun main() {
    println(sum(1, 2))
    // 3
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-simple-function-before"}
```

以下のように短縮できます。

中括弧 `{}` を削除し、代入演算子 `=` を使って関数の本体を宣言することができます。代入演算子 `=` を使用する場合、Kotlin では型推論が行われるため、戻り値の型を省略することもできます。これにより、`sum()` 関数は 1 行になります：

```
fun sum(x: Int, y: Int) = x + y

fun main() {
    println(sum(1, 2))
    // 3
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-simple-function-after"}
```

ただし、他の開発者にコードをすぐに理解してもらいたい場合は、代入演算子 `=` を使用する場合でも、戻り値の型を明示的に定義しておくことをお勧めします。

関数の本体を `{}` の波括弧を使って宣言する場合、戻り値の型が `Unit` 型でない限り、戻り値の型を宣言する必要があります。

{style="note"}

Early returns in functions

関数内での処理の切上げ

関数内のコードを特定のポイント以降で処理を中断するには、`return` キーワードを使用します。この例では、条件式が真であると判定された場合に、`if` を使用して関数から早期に返却しています：

```
// 登録済みユーザー名のリスト
val registeredUsernames = mutableListOf("john_doe", "jane_smith")

// 登録済みメールアドレスの一覧
val registeredEmails = mutableListOf("john@example.com", "jane@example.com")

fun registerUser(username: String, email: String): String {
    // ユーザー名がすでに使用されている場合は、途中で処理を切上げ
    if (username in registeredUsernames) {
        return "Username already taken. Please choose a different username."
    }

    // そのメールアドレスがすでに登録済みの場合は、途中で処理を切上げ
    if (email in registeredEmails) {
        return "Email already registered. Please use a different email."
    }

    // ユーザー名とメールアドレスがすでに登録されていない場合は、登録を続行
}
```

```

        registeredUsernames.add(username)
        registeredEmails.add(email)

        return "User registered successfully: $username"
    }

    fun main() {
        println(registerUser("john_doe", "newjohn@example.com"))
        // Username already taken. Please choose a different username.
        println(registerUser("new_user", "newuser@example.com"))
        // User registered successfully: new_user
    }

```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-function-early-return"}

Functions practice

関数の演習

Exercise 1 {initial-collapse-state="collapsed" collapsible="true" id="functions-exercise-1"}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="functions-exercise-1"}

`circleArea` という関数を記述してください。この関数は、整数形式の円の半径をパラメータとして受け取り、その円の面積を出力します。

この演習では、`PI` を通じて 円周率 の値にアクセスできるように、パッケージをインポートします。パッケージのインポートに関する詳細については、[パッケージとインポート](#)を参照してください。

{style="tip"}

円の面積を計算する式は、 πr^2 であり、ここでは半径である。

|---|---|

```

import kotlin.math.PI

// Write your code here

fun main() {
    println(circleArea(2))
}

```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-functions-exercise-1"}

|---|---|

```
import kotlin.math.PI

fun circleArea(radius: Int): Double {
    return PI * radius * radius
}

fun main() {
    println(circleArea(2)) // 12.566370614359172
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-functions-solution-1"}

Exercise 2 {initial-collapse-state="collapsed" collapsible="true" id="functions-exercise-2"}

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="functions-exercise-2"}

前の演習で作成した `circleArea` 関数を、単一式関数として書き直してください。

|---|---|

```
import kotlin.math.PI

// Write your code here

fun main() {
    println(circleArea(2))
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-functions-exercise-2"}

|---|---|

```
import kotlin.math.PI

fun circleArea(radius: Int): Double = PI * radius * radius

fun main() {
    println(circleArea(2)) // 12.566370614359172
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-functions-solution-2"}

Exercise 3 {initial-collapse-state="collapsed" collapsible="true" id="functions-exercise-3"}

課題 3 {initial-collapse-state="collapsed" collapsible="true" id="functions-exercise-3"}

時間、分、秒で指定された時間間隔を秒に変換する関数があります。ほとんどの場合、関数の引数は1つか2つだけ渡せばよく、残りは0に設定します。デフォルトのパラメータ値や名前付き引数を使用して、関数とその呼び出しコードを改善し、コードの可読性を高めてください。

|---|---|

```
fun intervalInSeconds(hours: Int, minutes: Int, seconds: Int) =
    ((hours * 60) + minutes) * 60 + seconds

fun main() {
    println(intervalInSeconds(1, 20, 15))
    println(intervalInSeconds(0, 1, 25))
    println(intervalInSeconds(2, 0, 0))
    println(intervalInSeconds(0, 10, 0))
    println(intervalInSeconds(1, 0, 1))
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-functions-exercise-3"}

|---|---|

```
fun intervalInSeconds(hours: Int = 0, minutes: Int = 0, seconds: Int = 0) =
    ((hours * 60) + minutes) * 60 + seconds

fun main() {
    println(intervalInSeconds(1, 20, 15))
    println(intervalInSeconds(minutes = 1, seconds = 25))
    println(intervalInSeconds(hours = 2))
    println(intervalInSeconds(minutes = 10))
    println(intervalInSeconds(hours = 1, seconds = 1))
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-functions-solution-3"}

Lambda expressions

ラムダ式

Kotlin では、ラムダ式を使用することで、関数のコードをさらに簡潔に書くことができます。

たとえば、次のような `uppercaseString()` 関数：

```
fun uppercaseString(text: String): String {
    return text.uppercase()
}
```

```
}  
fun main() {  
    println(uppercaseString("hello"))  
    // HELLO  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-function-before"}

ラムダ式として記述することもできます：

```
fun main() {  
    val upperCaseString = { text: String -> text.uppercase() }  
    println(upperCaseString("hello"))  
    // HELLO  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-variable"}

ラムダ式は一見すると理解しにくい場合があるので、詳しく見ていきましょう。ラムダ式は中括弧 `{ }` で囲んで記述します。

ラムダ式の中では、次のように記述します：

- `->` が後に続くパラメータ。
- `->` の後の関数本体。

前の例では：

- `text` は関数の引数です。
- `text` の型は `String` です。
- この関数は、`text` に対して呼び出された `.uppercase()` 関数の結果を返します。
- ラムダ式全体が、代入演算子 `=` を用いて `upperCaseString` 変数に代入されます。
- ラムダ式は、変数 `upperCaseString` を関数のように扱い、文字列 `「hello」` を引数として渡すことで呼び出されます。
- `println()` 関数は結果を出力します。

パラメータのないラムダ式を宣言する場合、`->` を使用する必要はありません。例えば：

```
{ println("Log message") }
```

{style="note"}

ラムダ式はさまざまな方法で使用できます。たとえば、次のような使い方ができます：

- ラムダ式を別の関数のパラメータとして渡す
- 関数からラムダ式を返す
- ラムダ式を単独で呼び出す

Pass to another function

別の関数に渡す

関数にラムダ式を渡すことが役立つ典型的な例として、コレクションに対して `.filter()` 関数を使用する場合があります：

```
fun main() {
    //sampleStart
    val numbers = listOf(1, -2, 3, -4, 5, -6)

    val positives = numbers.filter ({ x -> x > 0 })
    // ラムダ式{ x -> x > 0 }が、条件式

    val isNegative = { x: Int -> x < 0 } // このラムダ式が、条件式
    val negatives = numbers.filter(isNegative)

    println(positives)
    // [1, 3, 5]
    println(negatives)
    // [-2, -4, -6]
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-filter"}

`.filter()` 関数は、条件式としてラムダ式を受け取り、それをリストの各要素に適用します。この関数は、条件式が `true` を返した場合にのみ、その要素を残します：

- `{ x -> x > 0 }` は、要素が正の場合に `true` を返します。
- `{ x -> x < 0 }` は、要素が負の数である場合に `true` を返します。

この例では、ラムダ式を関数に渡す 2 つの方法を示しています：

- 正の数については、この例では `.filter()` 関数内でラムダ式を直接追加しています。
- 負の数については、この例ではラムダ式を `isNegative` 変数に代入しています。その後、`isNegative` 変数が `.filter()` 関数の引数として使用されます。この場合、ラムダ式内で関数引数 (`x`) の型を指定する必要があります。

関数の引数がラムダ式のみの場合は、関数の括弧 `()` を省略できます：

```
val positives = numbers.filter { x -> x > 0 }
```

これは [トレーリング・ラムダ](#) の例であり、これについてはこの章の最後で詳しく説明します。

{style="note"}

もうひとつの良い例として、`.map()` 関数を使ってコレクション内の要素を変換する方法があります：

```
fun main() {
    //sampleStart
    val numbers = listOf(1, -2, 3, -4, 5, -6)
    val doubled = numbers.map { x -> x * 2 }

    val isTripled = { x: Int -> x * 3 }
    val tripled = numbers.map(isTripled)

    println(doubled)
    // [2, -4, 6, -8, 10, -12]
    println(tripled)
    // [3, -6, 9, -12, 15, -18]
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-map"}

`.map()` 関数は、変換関数としてラムダ式を受け取ります：

- `{ x -> x * 2 }` は、リストの各要素を取り出し、その要素に 2 を掛けた値を返します。
- `{ x -> x * 3 }` は、リストの各要素を取り出し、その要素に 3 を掛けた値を返します。

Function types

関数型

関数からラムダ式を返すには、まず**関数型**について理解しておく必要があります。

基本的な型についてはすでに学習しましたが、関数自体にも型があります。Kotlinの型推論では、パラメータの型から関数の型を推論することができます。しかし、関数の型を明示的に指定する必要がある場合もあるかもしれません。コンパイラは、その関数で何が許可され、何が許可されないかを判断するために、関数の型を必要とします。

関数型の構文は次のとおりです：

- 各パラメータの型は、括弧 `()` 内に記述し、コンマ `,` で区切ります。
- `->` の後に記述される戻り値の型。

例：`(String) -> String` や `(Int, Int) -> Int`。

`uppercaseString()` の関数型が定義されている場合、ラムダ式は次のようになります：

```
val uppercaseString: (String) -> String = { text -> text.uppercase() }

fun main() {
    println(uppercaseString("hello"))
    // HELLO
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-function-type"}
```

ラムダ式に引数がない場合は、括弧 `()` を空にします。例：`() -> Unit`

パラメータ型および戻り値型は、ラムダ式内か、関数型として宣言する必要があります。そうしないと、コンパイラはあなたのラムダ式がどのような型なのかを判別できなくなります。

たとえば、次のようなコードは動作しません：

```
val upperCaseString = { str -> str.uppercase() }
```

{style="note"}

Return from a function

関数からの戻り値

関数からラムダ式を返すことができます。ラムダ式が返した型の種類を、コンパイラが認識できるようにするには、関数型を宣言する必要があります。

次の例では、`toSeconds()` 関数の関数型は `(Int) -> Int` となっています。これは、この関数が常に、型 `Int` の引数を受け取り、`Int` 型の値を返すラムダ式を返すためです。

この例では、`when` 式を使用して、`toSeconds()` が呼び出された際にどのラムダ式が返されるかを決定しています：

(訳者注：次の例では`toSeconds`関数が`time`に与えられた文字列に従い、ラムダ式、例えば`{ value -> value * 60 * 60 }`などを戻している。)

```
// ラムダ式を返す関数
fun toSeconds(time: String): (Int) -> Int = when (time) {
    "hour" -> { value -> value * 60 * 60 }
    "minute" -> { value -> value * 60 }
    "second" -> { value -> value }
    else -> { value -> value }
}

fun main() {
    val timesInMinutes = listOf(2, 10, 15, 1)
    val min2sec = toSeconds("minute")
    val totalTimeInSeconds = timesInMinutes.map(min2sec).sum()
    println("Total time is $totalTimeInSeconds secs")
    // Total time is 1680 secs
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-return-from-function"}
```

Invoke separately

個別に呼び出す

Lambda expressions can be invoked on their own by adding parentheses `()` after the curly braces `{}` and including any parameters within the parentheses:

```
fun main() {  
    //sampleStart  
    println({ text: String -> text.uppercase() }("hello"))  
    // HELLO  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-standalone"}

Trailing lambdas

トレイリング・ラムダ（末尾のラムダ式）

すでにご覧になった通り、関数の引数がラムダ式のみの場合は、関数の括弧 `()` を省略することができます。また関数の最後の引数としてラムダ式が渡される場合、そのラムダ式は関数の括弧 `()` の外側に記述することができます。いずれの場合も、この構文は **トレイリング・ラムダ**（末尾のラムダ式）と呼ばれます。

たとえば、`.fold()` 関数は、初期値（initial）とラムダ式（演算）を受け取ります：

```
fun main() {  
    //sampleStart  
    // 初期値はゼロです。  
    // この演算では、初期値とリスト内の各要素を累積的に足し合わせます。  
    println(listOf(1, 2, 3).fold(10, { x, item -> x + item })) // 16  
  
    // あるいは、トレイリング・ラムダ式の形で  
    println(listOf(1, 2, 3).fold(10) { x, item -> x + item }) // 16  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-trailing-lambda"}

ラムダ式に関する詳細については、[ラムダ式と匿名関数](#)を参照してください。

このツアーの次のステップでは、Kotlinの[クラス](#)について学びます。

Lambda expressions practice {completion-point="true"}

ラムダ式の演習 {completion-point="true"}

Exercise 1 {initial-collapse-state="collapsed" collapsible="true" id="lambdas-exercise-1"}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="lambdas-exercise-1"}

Webサービスでサポートされているアクションの一覧(actions)、すべてのリクエストに共通するプレフィックス(prefix)、および特定のリソースのID(id)があります。IDが5のリソースに対して、例えばアクションtitleを実行するには、次のURLを作成する必要があります：

`https://example.com/book-info/5/title.`

ラムダ式を使用して、アクションのリストからURLのリストを作成しなさい。

|---|---|

```
fun main() {
    val actions = listOf("title", "year", "author")
    val prefix = "https://example.com/book-info"
    val id = 5
    val urls = // Write your code here
    println(urls)
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambdas-exercise-1"}

|---|---|

```
fun main() {
    val actions = listOf("title", "year", "author")
    val prefix = "https://example.com/book-info"
    val id = 5
    val urls = actions.map { action -> "$prefix/$id/$action" }
    println(urls) // [https://example.com/book-info/5/title,
https://example.com/book-info/5/year, https://example.com/book-info/5/author]
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-lambdas-solution-1"}

Exercise 2 {initial-collapse-state="collapsed" collapsible="true" id="lambdas-exercise-2"}

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="lambdas-exercise-2"}

Int型の値とアクション（型が () -> Unit の関数）を受け取り、そのアクションを指定された回数だけ繰り返す関数を作成してください。その後、この関数を使って「Hello」を5回出力してください。

|---|---|

```
fun repeatN(n: Int, action: () -> Unit) {
    // Write your code here
}
```

```
fun main() {  
    // Write your code here  
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambdas-exercise-2"}

|---|---|

```
fun repeatN(n: Int, action: () -> Unit) {  
    for (i in 1..n) {  
        action()  
    }  
}  
  
fun main() {  
    repeatN(5) {  
        println("Hello")  
    }  
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-lambdas-solution-2"}

- [Previous step](#)
- [Next step](#)